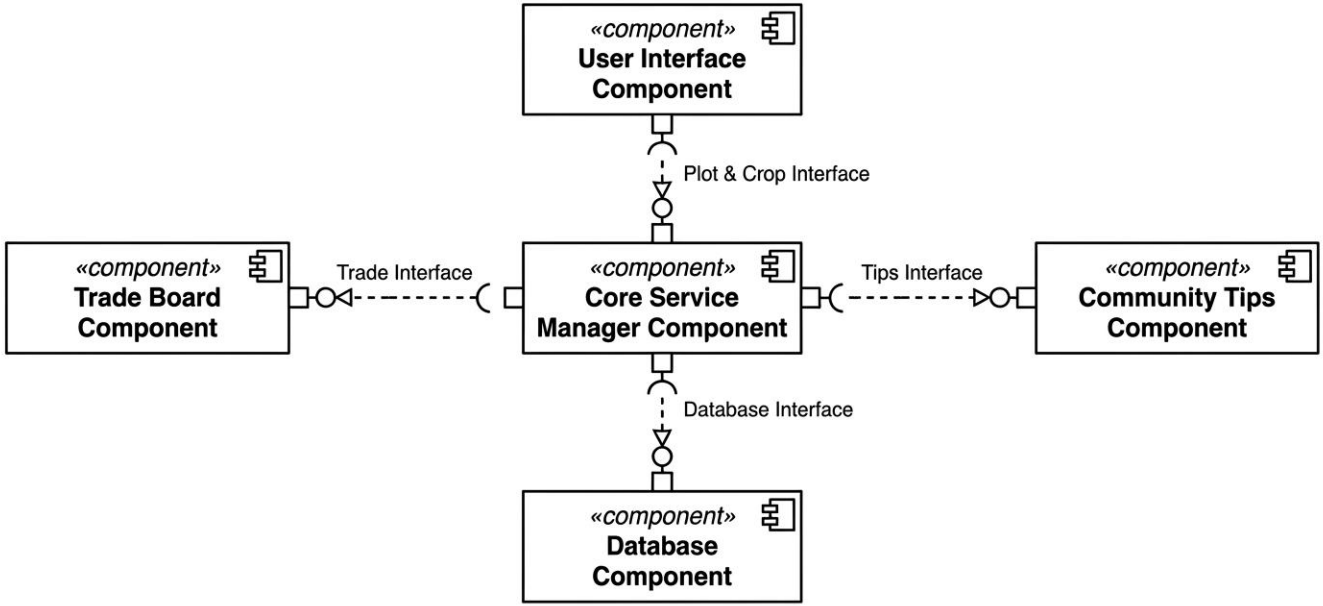
Urban Rooftop Gardening Co-op App — Component Diagram (Layered Architecture)



Architecture Justification

Urban Rooftop Gardening Co-op App, Problem Statement #65

Architecture Selection

I chose the Layered Architecture for the Urban Rooftop Gardening Co-op App system.

Two Reasons

First, the app's functionality naturally splits into three clear concerns: a presentation layer for gardener and Community Leader interactions, a business layer that handles plot registration, crop logging, trade listings, and tip sharing, and a data layer that persists everything including offline-cached entries. A layered structure keeps these concerns cleanly separated without adding the operational overhead of running and coordinating independent services, which isn't justified for a single community-scale app with a modest user base.

Second, several features depend directly on one another in a straightforward top-down way, a gardener's action in the UI triggers a business rule (like validating a harvest date or checking plot approval status), which then reads or writes to the database. This linear flow maps naturally onto layered architecture's request-passes-down, response-passes-up model, and it keeps the Core Service Manager component central and easy to reason about as the one place all business rules live.

Security Advantage

Because all requests must pass through the business layer before reaching the data layer, the Core Service Manager acts as a single enforcement point for access control, for example, confirming a plot is Community Leader-approved before allowing trade listings, or ensuring only a plot's owner can edit its crop logs. The presentation layer never talks to the database directly, so there's no way for the UI to bypass validation and write unauthorized data straight to storage.

Performance Benefit

Because business logic and validation live in one layer rather than being distributed across independently-deployed services, requests avoid the network hops and inter-service calls that a microservices approach would introduce, each action, like registering a plot or listing surplus produce, resolves in-process through a single, direct call chain from UI to business logic to database, keeping response times fast and predictable even as the number of gardeners using the app grows.